Reconstrucción temporal latente (LSTM-AE) y discriminación adversarial (LSTM–GAN) en la detección sin etiquetas de comportamientos maliciosos en series temporales de logs de servidores universitarios

Arturo Ruiz Tarazona
Facultad de Ingeniería Eléctrica y Electrónica (FIEE), Escuela Profesional de Ingeniería de Ciberseguridad
Universidad Nacional de Ingeniería (UNI), Lima, Perú

Los registros (*logs*) de servidores universitarios crecen a un ritmo que impide su revisión manual y carecen de etiquetas para entrenar detectores supervisados. Este estudio evalúa, sin etiquetas, si la reconstrucción temporal latente de un autocodificador LSTM (LSTM-AE) mejora la detección de anomalías frente a Isolation Forest, y si añadirle un discriminador adversarial que compite con él en un juego *minimax* (LSTM–GAN) aporta mejoras adicionales. Se empleó un diseño cuasi-experimental, transversal y comparativo, con medida pre-prueba/post-prueba sobre datos crudos: 3 217 544 eventos de PostgreSQL, GitLab y un ERP en Odoo, agrupados en 53 416 ventanas de cinco minutos mediante Drain3 y Word2Vec. Los modelos se entrenaron sin etiquetas y se evaluaron con etiquetas débiles derivadas de reglas de seguridad, con partición cronológica y diez semillas por subconjunto (30 pares). El modelo de reconstrucción temporal LSTM-AE incrementó el puntaje F1 de 25.06 % a 33.56 % ($p < .001$) y redujo la tasa de falsos positivos de 6.27 % a 3.28 % ($p < .001$); además, elevó el AUC-ROC a 89.04 %, por encima de la variante LSTM–GAN (85.68 %; $p < .001$). El discriminador no mejoró el F1 ($p = .129$) y aumentó la variabilidad entre semillas. Se concluye que la reconstrucción temporal de series semánticas sostiene las ganancias de detección, mientras que la adición de un discriminador adversarial no aporta beneficios métricos y disminuye la estabilidad del modelo.

Palabras clave: detección de anomalías, registros del sistema, autocodificador LSTM, red generativa adversarial, aprendizaje no supervisado, ciberseguridad

Ideas destacadas

- Un autocodificador LSTM (LSTM-AE) detecta, sin etiquetas, inyecciones SQL y escaneos web en registros universitarios reales.
- Frente a Isolation Forest, el LSTM-AE eleva el F1 en 8.5 puntos y reduce las falsas alarmas en un 48 %, con significancia estadística en 30 pares.
- Añadir un discriminador adversarial (LSTM–GAN) no mejora el F1, reduce el AUC en 3.4 puntos y vuelve al modelo más inestable.

1. Introducción

Los sistemas informáticos registran de forma continua su actividad en archivos de *logs*, que constituyen la principal fuente de evidencia para detectar fallos, usos indebidos e intrusiones. El volumen de estos registros en infraestructuras modernas hace inviable su inspección manual, y las reglas o firmas escritas a mano solo reconocen escenarios previstos de antemano, por lo que se requieren técnicas que aprendan automáticamente el comportamiento normal y señalen sus desviaciones (Duan et al., 2021). Además, en infraestructuras con varias fuentes de *logs*, las anomalías pueden ocultarse en secuencias de operaciones aparentemente válidas, lo que obliga a modelar las dependencias temporales entre eventos (Luo et al., 2025). En una universidad, un mismo conjunto de servidores aloja bases de datos académicas y administrativas, plataformas de control de versiones y aplicaciones de gestión expuestas a Internet, de modo que una intrusión en cualquiera de ellos compromete información sensible y la continuidad de los servicios.

La variable dependiente de este estudio es la *detección de comportamientos maliciosos*, cuya problemática presenta tres rasgos. Primero, las anomalías son raras y no se conocen de antemano, por lo que rara vez existen etiquetas para entrenar clasificadores supervisados; obtenerlas a escala industrial es costoso y debe repetirse cuando cambia el comportamiento normal (Albert, 2025). Segundo, los métodos no supervisados tienden a producir un número elevado de falsos positivos, lo que satura a los operadores y reduce la confianza en las alertas; por ello, minimizar las ventanas normales señaladas como anómalas es un objetivo explícito de los detectores no supervisados de *logs* (Farzad & Gulliver, 2020).

Tercero, buena parte de los resultados publicados se obtienen sobre unos pocos conjuntos de referencia previamente etiquetados (HDFS, BGL, Thunderbird, OpenStack), con supuestos optimistas como entrenar solo con datos normales o ajustar umbrales sobre el propio conjunto de prueba (Albert, 2025). A ello se suma que, en series temporales reales, los datos etiquetados son difíciles de obtener y el desbalance entre clases complica fijar el umbral de decisión (Yokkampon et al., 2022). Por último, propuestas recientes combinan el aprendizaje profundo con algoritmos clásicos como Isolation Forest (Farzad & Gulliver, 2020), de modo que toda arquitectura compleja debe contrastarse con una línea base tradicional.

Las arquitecturas basadas en reconstrucción ofrecen una alternativa sin etiquetas: un autocodificador aprende a reconstruir el comportamiento normal y las entradas que no logra reconstruir se consideran anómalas (Aslam et al., 2024; Gorokhov et al., 2023). Su combinación con redes generativas adversariales (GAN), en las que un discriminador obliga al generador a producir reconstrucciones más fieles, ha mostrado buenos resultados en series temporales de sensores y de Internet de las cosas (Niu et al., 2020; Qian et al., 2025). Sin embargo, **el vacío** que aborda este trabajo es la ausencia de evidencia sobre cuánto aportan, por separado, la reconstrucción temporal y el discriminador adversarial en registros *crudos y heterogéneos* de infraestructura universitaria real, sin etiquetas, con partición cronológica y en comparación estadística con una técnica tradicional.

De ahí surge el **problema general**: ¿en qué medida la reconstrucción temporal latente (LSTM-AE) y la adición de un discriminador adversarial (LSTM–GAN) mejoran la detección sin etiquetas de comportamientos maliciosos en las series temporales de *logs* de la infraestructura de servidores universitarios frente a una línea base tradicional? El **propósito** u objetivo general es evaluar en qué medida ambos componentes mejoran esa detección frente a Isolation Forest, técnica no supervisada tradicional. Los **objetivos específicos** son (OE1) determinar en qué medida el LSTM-AE mejora la exactitud de la detección, medida con el puntaje F1, frente a Isolation Forest; (OE2) determinar en qué medida el LSTM-AE reduce la tasa de falsas alarmas (FPR) frente a Isolation Forest; y (OE3) determinar si la adición del discriminador adversarial (LSTM–GAN) mejora la exactitud y la capacidad discriminante (AUC-ROC) respecto del LSTM-AE. La hipótesis general sostiene que la reconstrucción temporal latente (LSTM-AE) mejora significativamente la exactitud y reduce las falsas alarmas frente a la línea base, mientras que el componente adversarial no genera mejoras estadísticamente significativas en exactitud y degrada la capacidad discriminante. De ella se derivan tres hipótesis específicas: el LSTM-AE mejora significativamente la exactitud de la detección frente a Isolation Forest (H1); el LSTM-AE reduce significativamente la tasa de falsas alarmas frente a Isolation Forest (H2); y la adición del discriminador (LSTM–GAN) no mejora significativamente la exactitud y reduce la capacidad discriminante respecto del LSTM-AE (H3). La **motivación** es práctica: los equipos de seguridad universitarios disponen de abundantes registros pero no de analistas para etiquetarlos. La **importancia** radica en aportar un experimento reproducible con datos reales, que cuantifica por separado el efecto de la reconstrucción temporal y el del componente adversarial. El **aporte** incluye además una representación de ventanas de cinco minutos como series semánticas que corrige la debilidad de la reconstrucción secuencial ante ráfagas de ataque repetitivas.

2. Revisión de la literatura

Duan et al. (2021) aplicaron una GAN a la detección de anomalías en *logs*, motivados por la escasez de anomalías reales y de conjuntos etiquetados con precisión, y por la insuficiencia de las reglas definidas por expertos. Su generador es un codificador-decodificador LSTM que produce plantillas de *log* a partir de palabras clave, y su discriminador, una red convolucional que distingue las plantillas generadas de las reales. Con ese esquema alcanzaron una precisión media del 95 % en datos reales. Ese trabajo, sin embargo, no se aplicó a registros crudos de infraestructura universitaria.

Albert (2025) demostró que los métodos semisupervisados del estado del arte colapsan cuando el conjunto de entrenamiento contiene anomalías mezcladas, condición habitual en datos industriales sin depurar. También mostró que la división cronológica, frente a la aleatoria, reduce drásticamente el F1 reportado: en BGL, de 92.7 % a 42.6 % con DeepLog. Esto evidencia que la literatura sobreestima el desempeño esperable en producción.

En el plano arquitectónico, Niu et al. (2020) propusieron un VAE-GAN en el que redes LSTM actúan como codificador, generador y discriminador, entrenados de forma conjunta, e identificaron las anomalías combinando la diferencia de reconstrucción con el resultado del discriminador. Qian et al. (2025) integraron autocodificadores y GAN en RGAnomaly y combinaron la pérdida de reconstrucción con la de discriminación en una puntuación única de anomalía. Ambos trabajos se validaron con series de sensores, no con registros de texto.

Nasirzadeh et al. (2025) alcanzaron un F1 medio de 99.61 % en siete conjuntos de *logs* de sistemas operativos, pero con un enfoque supervisado que requiere etiquetas. En síntesis, la literatura evidencia el potencial de los modelos de reconstrucción y de su variante adversarial, pero no responde si cada componente aporta valor, por separado, frente a técnicas tradicionales en registros universitarios reales, sin etiquetas y evaluados con rigor cronológico. Ese es el vacío que aborda este estudio.

3. Metodología

3.1. Enfoque, tipo y diseño

El estudio es cuantitativo, de tipo aplicado y de nivel correlacional-causal, porque busca establecer el efecto de la variable independiente, la arquitectura de reconstrucción temporal latente en sus variantes LSTM-AE y LSTM–GAN, sobre la variable dependiente, la detección de comportamientos maliciosos. El diseño es cuasi-experimental, transversal y comparativo, con medida pre-prueba/post-prueba. Es cuasi-experimental porque se aplica un estímulo sin asignación aleatoria de las unidades; transversal, porque los datos corresponden a un único periodo de observación del repositorio; y comparativo, porque se contrastan dos tratamientos sobre las mismas unidades. La pre-prueba corresponde al desempeño de la técnica tradicional (Isolation Forest). El estímulo es la implementación de dos tratamientos: el LSTM-AE, que aplica solo la reconstrucción temporal, y el LSTM–GAN, que le añade el discriminador adversarial. La post-prueba es el desempeño de ambos sobre idénticas ventanas de evaluación. Isolation Forest se eligió como línea base porque es un algoritmo no supervisado de uso consolidado en la detección de anomalías en *logs* (Farzad & Gulliver, 2020). Se eligió este diseño porque permite atribuir las diferencias al modelo y no a la muestra, y porque contrastar un modelo propuesto con líneas de base sobre los mismos datos es la práctica habitual del área (Nasirzadeh et al., 2025; Zhao et al., 2025). La comparación entre ambos tratamientos aísla la contribución del componente adversarial.

3.2. Población, muestra y unidad de análisis

La población es finita y está constituida por la totalidad de eventos de las series temporales de *logs crudos* del repositorio institucional *uni-logs*, publicado por el docente (https://uni-logs.us-mia-1.linodeobjects.com/list.html). El repositorio agrupa cuatro subconjuntos: `logdb`, `loggit`, `logsapp` y `logserver`. La muestra, no probabilística por conveniencia, comprende los tres subconjuntos procesados íntegramente en esta etapa, que se describen en la Tabla 1. El subconjunto `logserver` no se incorporó a la muestra porque su volumen, de aproximadamente 2.4 GB, excedía la capacidad de cómputo disponible para el entrenamiento.

- `logdb`: servidor PostgreSQL 12.
- `loggit`: plataforma GitLab, con registros de acceso de nginx, `sshd` y `gitlab-shell`.
- `logsapp`: ERP CubicERP basado en Odoo, con registros del servidor principal, del servidor *gevent* y de gunicorn.

Se excluyeron los registros de instalación del sistema operativo (`dpkg`, `apt`, `bootstrap`) por no reflejar la operación del servicio. La unidad de análisis es la ventana temporal fija de cinco minutos ($\Delta t = 5$ min), y la muestra suma 3 217 544 eventos agrupados en 53 416 ventanas.

3.3. Técnicas e instrumentos de recolección de datos

Se emplearon dos técnicas: la observación estructurada, basada en la minería de los *logs* crudos sin etiquetar, y la simulación mediante experimentación computacional, que consiste en entrenar y evaluar los modelos sin supervisión. El instrumento es un registro de observación estructurada, incluido en el documento de sistematización del estudio y en el repositorio de código. Lo componen una ficha técnica y una matriz pre-prueba/post-prueba que consigna, por fuente de *logs*, los valores de F1, FPR y AUC, su diferencia absoluta y su variación relativa. El registro se llena de forma automatizada con un programa en Python que integra tres herramientas:

- El analizador de plantillas Drain3, que separa la parte constante de cada mensaje de sus parámetros variables mediante un árbol de profundidad fija (Nasirzadeh et al., 2025; Zhao et al., 2025).
- Un conjunto de reglas de seguridad que genera las etiquetas débiles de referencia (Tabla 2).
- Las bibliotecas PyTorch, scikit-learn y gensim, con las que se entrenan los modelos y se calculan los indicadores de cada corrida.

El etiquetado por reglas o términos indicativos tiene precedente en Nasirzadeh et al. (2025). Es pertinente aquí porque los registros no incluyen etiquetas. Las etiquetas *no* se usan para entrenar ningún modelo, solo para evaluarlos.

3.4. Procedimiento experimental

El protocolo se ejecutó en tres fases secuenciales automatizadas.

- **Fase pre-prueba (línea base).** Se leyeron los registros, se convirtieron todas las marcas de tiempo a UTC y se unieron las líneas de continuación de trazas y sentencias SQL. Las líneas de acceso web se codificaron como *método–primer segmento de ruta–extensión–estado HTTP–familia de agente*, para que Drain3 conservara la señal de ataque. Después, Drain3 se aplicó en línea, en orden cronológico, y los eventos se agruparon en ventanas fijas de cinco minutos, estrategia empleada cuando los registros no contienen identificadores de sesión (Almodovar et al., 2024). Cada subconjunto se dividió cronológicamente en 60 % para entrenamiento, 20 % para validación y 20 % para prueba. Así se evita la fuga de información futura señalada por Albert (2025). El conjunto de

Tabla 1

Caracterización de los subconjuntos de registros crudos analizados

Subconjunto	Fuente	Eventos	Plantillas	Ventanas	Periodo (UTC)	Ent./Val./Prueba	Maliciosas (E/V/P)
`logdb`	PostgreSQL 12	267 896	186	5 719	01/03/2026 – 12/05/2026	3 431 / 1 144 / 1 144	2 / 1 / 90
`loggit`	GitLab	56 135	121	3 398	30/10/2025 – 12/05/2026	2 038 / 680 / 680	38 / 28 / 21
`logsapp`	CubicERP (Odoo)	2 893 513	361	44 299	20/11/2025 – 12/05/2026	26 579 / 8 860 / 8 860	24 / 2 / 8
Total		3 217 544	668	53 416		32 048 / 10 684 / 10 684	64 / 31 / 119

Nota. Ventanas de cinco minutos con al menos un evento. Las ventanas maliciosas corresponden a etiquetas débiles (Tabla 2), usadas solo para evaluar. E = entrenamiento; V = validación; P = prueba.

Tabla 2

Reglas de seguridad para el etiquetado débil por ventana

Subconjunto	Una ventana se etiqueta como maliciosa si …
`logdb`	contiene ≥ 1 firma de inyección SQL (`sleep(`, `XOR(`, `OR n=n`, `UNION SELECT`) o ≥ 3 fallos de sondeo (base de datos o rol inexistente, autenticación fallida).
`loggit`	contiene ≥ 1 URL con carga de ataque (SQLi/XSS, *path traversal*, `.php`, `wp-*`, `.env`), ≥ 1 agente escáner, ≥ 3 fallos SSH o ≥ 10 respuestas HTTP 4xx.
`logsapp`	contiene ≥ 1 URL con carga de ataque o ≥ 10 respuestas HTTP 4xx.

Nota. Se excluye el código 499 (cierre de conexión por el cliente) porque no indica una petición errónea.

entrenamiento *no* se depuró, por lo que contiene 64 ventanas maliciosas; el escenario es, por tanto, genuinamente no supervisado. Sobre estas ventanas se entrenó Isolation Forest con diez semillas aleatorias por subconjunto y se obtuvieron $F1_{pre}$, FPR_{pre} y AUC_{pre} frente a las etiquetas débiles.

- **Fase de aplicación del estímulo.** El LSTM-AE y el LSTM–GAN se entrenaron sin etiquetas sobre las mismas ventanas de entrenamiento, con las mismas diez semillas y la configuración de la Tabla 3.

- **Fase post-prueba.** El umbral de decisión ρ de cada modelo se fijó en el percentil 95 de sus puntuaciones de validación, sin intervención del tramo de prueba (Albert, 2025; Qian et al., 2025). Con él se evaluaron las mismas ventanas de prueba, se calcularon $F1_{post}$, FPR_{post} y AUC_{post} y se consignaron en el registro de observación.

3.5. Indicadores y análisis estadístico

La exactitud se midió con el puntaje F1, la precisión y la exhaustividad; las falsas alarmas, con la FPR y la especificidad; y la capacidad discriminante, con el AUC-ROC y el coeficiente de correlación de Matthews (MCC) (Almodovar et al., 2024; Zhang et al., 2025):

$$F1 = \frac{2PR}{P+R}, \qquad FPR = \frac{FP}{FP+TN}. \tag{1}$$

El AUC-ROC es independiente del umbral, lo que lo hace adecuado para comparar puntuaciones continuas (Gorokhov et al., 2023). Para cada indicador se calcularon la media, la desviación estándar, la diferencia absoluta $\Delta = \text{post} - \text{pre}$ y la variación relativa. Los valores se interpretan con los niveles criteriales de la matriz de operacionalización del estudio:

- F1: bajo (< 70 %), medio (70–85 %) y alto (> 85 %).
- FPR: crítico (> 10 %), aceptable (5–10 %) y óptimo (< 5 %).
- AUC: débil (< 80 %), aceptable (80–90 %) y óptimo (> 90 %).

La prueba de Shapiro–Wilk rechazó la normalidad de las diferencias en F1, FPR y AUC ($p < .05$). Por ello, las hipótesis se contrastaron con la prueba de rangos con signo de Wilcoxon para muestras relacionadas, con $\alpha = .05$, sobre 30 pares (10 semillas × 3 subconjuntos) y por subconjunto (10 pares), con tamaño del efecto $r = Z/\sqrt{n}$. La regla de decisión es la siguiente. Para H1 y H2, la hipótesis nula H_0 establece que no hay diferencia entre el LSTM-AE e Isolation Forest; si $p < .05$ y la diferencia tiene la dirección esperada, se rechaza H_0. H3 compara el LSTM–GAN con el LSTM-AE y se acepta si la diferencia de F1 no es significativa ($p \geq .05$) y la de AUC es significativa y negativa ($p < .05$). Como análisis complementario, se estimó mediante *bootstrap* (2 000 remuestreos) el intervalo de confianza al 95 % de la diferencia de AUC a nivel de ventana.

El umbral en el percentil 95 de validación tiene una limitación que conviene explicitar. Dado el extremo desbalance de clases en subconjuntos como `logsapp`, donde solo 8 de 8 860 ventanas de prueba son maliciosas, ese umbral implica que alrededor del 5 % de las ventanas normales lo supere, es decir, unas 443 alertas frente a 8 positivos reales. Ello contrae artificialmente la precisión y el F1. Por ello, los resultados se complementan con el AUC-ROC y con intervalos de confianza que no dependen del umbral.

3.6. Comparación con metodologías similares

El preprocesamiento sigue la cadena análisis de plantillas, representación semántica y agrupación por ventanas de Zhao et al. (2025) y Zhang et al. (2025), que también emplean Drain y Word2Vec. El criterio de anomalía por error de reconstrucción en series temporales se basa en la formulación de Aslam et al. (2024) y Niu et al. (2020); no obstante, dichos trabajos validan sus modelos en señales físicas continuas, de sistemas de control industrial (SWaT) y de sensores, a diferencia del presente estudio, enfocado en eventos discretos de *logs* de servidores. Gorokhov et al. (2023) sí aplican la reconstrucción a *logs*, aunque sobre conjuntos de referencia etiquetados. La puntuación que combina reconstrucción y discriminación procede de Niu et al. (2020) y Qian et al. (2025).

El estudio se diferencia de esos trabajos en tres aspectos: usa datos crudos sin depurar en lugar de conjuntos de referencia etiquetados, aplica una partición estrictamente cronológica y somete el resultado a pruebas de significancia y a la comparación entre componentes, elementos que la literatura revisada señala como necesarios (Albert, 2025).

4. Desarrollo

Los modelos se organizan jerárquicamente en tres niveles de complejidad creciente. El primero es Isolation Forest, puntual y sin contexto temporal. El segundo es el LSTM-AE, que añade contexto temporal y reconstrucción. El tercero es el LSTM–GAN, que añade el discriminador adversarial. En lo que sigue, la matriz de estados ocultos del codificador se denota $H = (\mathbf{h}_1, \ldots, \mathbf{h}_W)$, y la recodificación de la serie reconstruida, $\tilde{H} = (\tilde{\mathbf{h}}_1, \ldots, \tilde{\mathbf{h}}_W)$. La Figura 1 resume el flujo.

4.1. Representación semántica de las ventanas

Cada plantilla k se representa mediante el promedio normalizado $\mathbf{e}_k \in \mathbb{R}^{300}$ de los vectores Word2Vec (*skip-gram*) de sus términos, entrenados sobre el vocabulario de plantillas sin usar etiquetas (Zhao et al., 2025). Cada ventana t con conteos $c_{t,k}$ se representa como

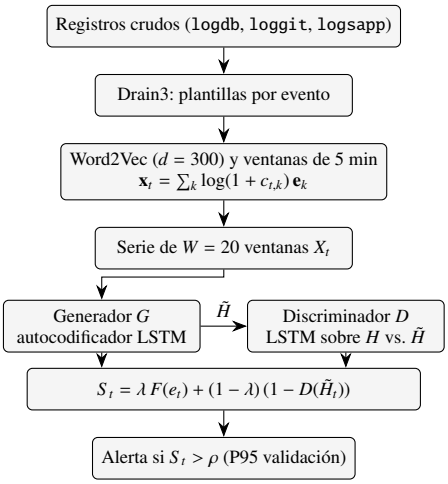
$$\mathbf{x}_t = \sum_k \log(1 + c_{t,k})\, \mathbf{e}_k, \tag{2}$$

estandarizado con la media y la desviación del tramo de entrenamiento. La entrada del modelo es la serie $X_t = (\mathbf{x}_{t-W+1}, \ldots, \mathbf{x}_t)$ con $W = 20$ (100 minutos) y paso 1, de modo que cada ventana recibe una puntuación propia.

Esta decisión corrige un problema detectado en pruebas preliminares. Al modelar secuencias de *eventos*, las ráfagas de ataque, compuestas por cientos de peticiones idénticas, resultaban trivialmente reconstruibles y obtenían puntuaciones bajas.

Figura 1

Arquitectura de reconstrucción temporal (LSTM-AE) y su variante adversarial (LSTM–GAN)



Nota. Elaboración propia. El LSTM-AE corresponde a la rama del generador sin el discriminador y puntúa con $F(e_t)$. $F(\cdot)$ es la función de distribución empírica del error de reconstrucción en validación.

4.2. Línea de base: Isolation Forest

Isolation Forest aísla observaciones mediante particiones aleatorias recursivas: las anomalías requieren menos particiones para quedar aisladas. Es una referencia no supervisada habitual en este dominio (Farzad & Gulliver, 2020; Nasirzadeh et al., 2025). Se usaron 200 árboles sobre el vector $\log(1 + c_{t,k})$ de cada ventana, y la puntuación de anomalía es la puntuación de aislamiento con signo invertido.

4.3. Autocodificador LSTM (LSTM-AE, generador G)

El codificador LSTM recorre la serie y la resume en el estado oculto final $\mathbf{z}_t = \mathbf{h}_W \in \mathbb{R}^{128}$:

$$\mathbf{h}_i = \mathrm{LSTM}_{enc}(\mathbf{x}_i, \mathbf{h}_{i-1}). \tag{3}$$

El decodificador recibe $\mathbf{z}_t$ en cada paso y reconstruye la serie, $\hat{\mathbf{x}}_i = W_o\, \mathrm{LSTM}_{dec}(\mathbf{z}_t, \mathbf{s}_{i-1}) + \mathbf{b}_o$. La pérdida de reconstrucción es el error cuadrático medio sobre las posiciones válidas m_i:

$$\mathcal{L}_{rec} = \frac{1}{\sum_i m_i} \sum_{i=1}^{W} m_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|_2^2. \tag{4}$$

4.4. Discriminador adversario secuencial

Como en Niu et al. (2020), el discriminador D es una LSTM, en este caso de 64 unidades y con salida sigmoide. A diferencia de ese trabajo, opera sobre representaciones latentes: recibe la secuencia de estados ocultos H de la entrada

real y la secuencia $\tilde{H} = \text{LSTM}_{enc}(\hat{X})$ obtenida al recodificar la reconstrucción, y aprende a distinguirlas. Su salida es $D(H) = \sigma(\mathbf{w}_d^\top \mathbf{h}_W^D + b_d)$, donde $\mathbf{h}_W^D$ es su último estado oculto. El generador y el discriminador compiten en el juego *minimax*

$$\min_G \max_D \; V(D,G) = \mathbb{E}_X[\log D(H)] + \mathbb{E}_X\Big[\log\big(1 - D(\tilde{H})\big)\Big], \tag{5}$$

que en la práctica se optimiza alternando dos pasos:

$$\mathcal{L}_D = -\Big[\log D(H) + \log\big(1 - D(\tilde{H})\big)\Big], \tag{6}$$

$$\mathcal{L}_G = \alpha\, \mathcal{L}_{rec} + (1-\alpha)\Big[-\log D(\tilde{H})\Big], \tag{7}$$

con $\alpha = 0.8$; el término adversarial de (7) es la forma no saturante de (5). La ecuación (7) obliga al generador a producir reconstrucciones cuya codificación latente sea indistinguible de la real.

4.5. Puntuación dual y decisión

Para cada ventana se combinan el error de reconstrucción de su última posición, $e_t = \|\mathbf{x}_t - \hat{\mathbf{x}}_t\|_2^2$, y el rechazo del discriminador (Qian et al., 2025):

$$S_t = \lambda\, F_{val}(e_t) + (1-\lambda)\big(1 - D(\tilde{H}_t)\big), \tag{8}$$

donde F_{val} es la función de distribución empírica del error en validación, que lleva ambos términos a la escala $[0, 1]$. El peso $\lambda = 0.7$ se fijó a priori, sin ajustarlo sobre los datos, para dar mayor peso a la reconstrucción que a la discriminación adversarial, cuya salida es menos estable por las dificultades propias del entrenamiento de las GAN (Qian et al., 2025); la alta variabilidad del discriminador observada después entre semillas (Sección 5.3) es coherente con esa elección. La ventana se clasifica como maliciosa si $S_t > \rho$. El LSTM-AE usa $S_t = F_{val}(e_t)$ y se entrena solo con la ecuación (4). La Tabla 3 resume la configuración.

5. Resultados

5.1. Desempeño por modelo y subconjunto

La Tabla 4 y la Figura 2 presentan los indicadores en el tramo de prueba. En el consolidado de los tres subconjuntos, el LSTM-AE obtuvo F1 = 33.56 %, FPR = 3.28 % y AUC = 89.04 %, frente a 25.06 %, 6.27 % y 86.52 % de Isolation Forest. Esto equivale a ΔF1 = +8.50 puntos porcentuales (+33.9 %) y ΔFPR = −2.98 puntos (−47.6 %). El LSTM–GAN obtuvo F1 = 32.26 %, FPR = 2.88 % y AUC = 85.68 %.

Por subconjunto, el LSTM-AE mejoró el F1 en `logdb` (+7.56), `loggit` (+15.97) y `logsapp` (+1.98), y redujo la FPR en los tres (−1.41, −0.44 y −7.09). Su AUC superó al de Isolation Forest en `logdb` (+3.74) y `logsapp` (+11.58), pero no en `loggit` (−7.74). Frente al LSTM-AE, el LSTM–GAN obtuvo un AUC menor en los tres subconjuntos.

Tabla 3

Configuración de los modelos

Parámetro	Valor
Dimensión Word2Vec / estado oculto LSTM	300 / 128
Ventanas por serie (W) / paso	20 / 1
Unidades del discriminador	64
α (peso de $\mathcal{L}_{rec}$) / λ (peso en S_t)	0.8 / 0.7
Optimizador / tasa de aprendizaje	Adam / 10^{-3}
Tamaño de lote / máx. secuencias por época	1 024 / 8 192
Épocas máximas / paciencia (parada temprana)	30 / 5
Árboles de Isolation Forest	200
Umbral ρ	P95 de validación
Semillas por subconjunto	10

Nota. El límite de 8 192 secuencias por época solo afecta a `logsapp`.

El F1 de `logsapp` es bajo para todos los modelos (< 3 %) porque solo 8 de 8 860 ventanas de prueba son maliciosas. Con el umbral en el percentil 95, el LSTM-AE emitió en promedio unas 477 falsas alarmas (FPR = 5.39 %) y el LSTM–GAN unas 261 (2.95 %) frente a esos 8 positivos reales. Ello contrae la precisión (1.43 % y 0.96 %) y el F1 (2.81 % y 1.85 %).

Según los niveles criteriales, el F1 consolidado de los tres modelos se ubica en el nivel bajo (< 70 %). La FPR consolidada pasa del nivel aceptable (6.27 %) al óptimo con el LSTM-AE (3.28 %) y con el LSTM–GAN (2.88 %). En `logsapp`, pasa del nivel crítico (12.49 %) al aceptable con el LSTM-AE (5.39 %) y al óptimo con el LSTM–GAN (2.95 %). El AUC consolidado se mantiene en el nivel aceptable (80–90 %) con los tres modelos, y en `logsapp` el LSTM-AE alcanza el nivel óptimo (96.54 %).

5.2. Contraste de hipótesis

La Tabla 5 muestra los contrastes de Wilcoxon. H1 se confirma: el LSTM-AE aumentó el F1 frente a Isolation Forest en el consolidado ($p < .001$, $r = .84$) y en los tres subconjuntos. H2 se confirma: la FPR se redujo en el consolidado ($p < .001$, $r = .87$) y en los tres subconjuntos. H3 se confirma: frente al LSTM-AE, el LSTM–GAN no mejoró el F1 ($\bar{\Delta} = -1.31$; $p = .129$) y redujo significativamente el AUC ($\bar{\Delta} = -3.36$; $p < .001$, $r = .81$), con caídas en los tres subconjuntos. Como resultado complementario, el AUC del LSTM-AE no difirió significativamente del de Isolation Forest en el consolidado ($p = .114$), porque fue superior en `logdb` y `logsapp` e inferior en `loggit`.

El *bootstrap* a nivel de ventana confirma estos patrones:

- LSTM-AE menos Isolation Forest: +3.63 puntos en `logdb`, IC 95 % [1.48, 5.93]; −8.22 en `loggit`,

Tabla 4

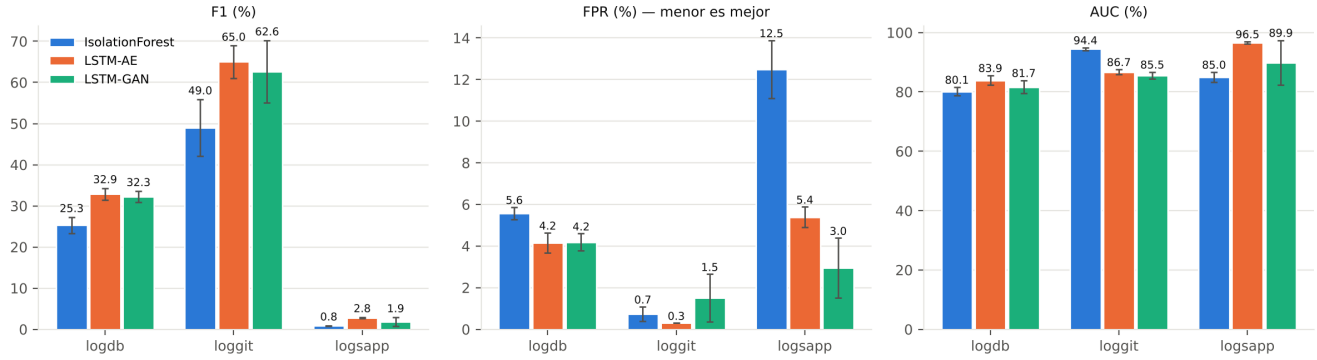
Indicadores de detección en el tramo de prueba (media ± desviación estándar de 10 semillas)

Subconjunto	Modelo	F1 (%)	Precisión (%)	Exhaustividad (%)	FPR (%)	AUC (%)	MCC
`logdb`	Isolation Forest	25.33 ± 1.96	26.86	24.00	5.57 ± 0.30	80.15 ± 1.40	.19
	LSTM-AE	32.89 ± 1.44	37.70	29.22	4.16 ± 0.49	83.88 ± 1.63	.28
	LSTM–GAN	32.28 ± 1.34	37.02	28.67	4.18 ± 0.42	81.67 ± 2.17	.28
`loggit`	Isolation Forest	49.03 ± 6.88	64.30	40.00	0.74 ± 0.35	94.44 ± 0.43	.49
	LSTM-AE	65.00 ± 3.96	84.65	52.86	0.30 ± 0.00	86.70 ± 0.80	.66
	LSTM–GAN	62.64 ± 7.61	63.71	65.24	1.50 ± 1.15	85.50 ± 1.18	.62
`logsapp`	Isolation Forest	0.82 ± 0.12	0.41	57.50	12.49 ± 1.40	84.97 ± 1.67	.04
	LSTM-AE	2.81 ± 0.15	1.43	86.25	5.39 ± 0.49	96.54 ± 0.36	.11
	LSTM–GAN	1.85 ± 1.09	0.96	33.75	2.95 ± 1.43	89.88 ± 7.51	.05
Consolidado	Isolation Forest	25.06	30.53	40.50	6.27	86.52	.24
	LSTM-AE	33.56	41.26	56.11	3.28	89.04	.35
	LSTM–GAN	32.26	33.90	42.55	2.88	85.68	.32

Nota. Umbral ρ en el percentil 95 de validación. FPR: menor es mejor.

Figura 2

F1, tasa de falsos positivos y AUC por modelo y subconjunto



Nota. Barras: media de 10 semillas; líneas de error: desviación estándar.

IC 95 % [−18.84, −0.30]; y +11.81 en `logsapp`, IC 95 % [5.71, 19.80].

- LSTM–GAN menos LSTM-AE: −1.91 en `logdb`, IC 95 % [−2.79, −1.12]; −0.70 en `loggit`, IC 95 % [−2.95, 0.76]; y −3.35 en `logsapp`, IC 95 % [−4.92, −1.44].

Las curvas ROC se presentan en la Figura 3.

5.3. Contribución del discriminador y sensibilidad al umbral

Al retirar el discriminador (LSTM-AE), no hubo diferencias significativas en $F1$ ($p = .129$) ni en FPR ($p = .311$). El AUC, en cambio, mejoró en 3.36 puntos ($p < .001$, $r = .81$) y el MCC fue significativamente mayor ($p = .002$). Asimismo, el LSTM–GAN presentó mayor inestabilidad operacional: la desviación de su AUC en `logsapp` fue de 7.51 puntos, frente a 0.36 del LSTM-AE.

Al elevar el umbral al percentil 99, el LSTM-AE mantuvo un F1 de 34.53 % en `logdb`, con FPR = 1.69 %, mientras que Isolation Forest descendió a 19.96 %. En `loggit`, el LSTM-AE obtuvo 35.71 % frente a 30.60 % de Isolation Forest, y el LSTM–GAN conservó más exhaustividad con ese umbral (F1 = 44.39 %). La ventaja frente a la línea base, por tanto, no depende de un umbral particular.

5.4. Hallazgos cualitativos

La Figura 4 muestra la puntuación del LSTM-AE en el tramo de prueba. El modelo detectó, sin etiquetas, campañas reales de ataque:

- Inyecciones SQL ciegas basadas en tiempo (`sleep()`, `XOR`) contra PostgreSQL, el 1 y el 2 de mayo de 2026.

Tabla 5

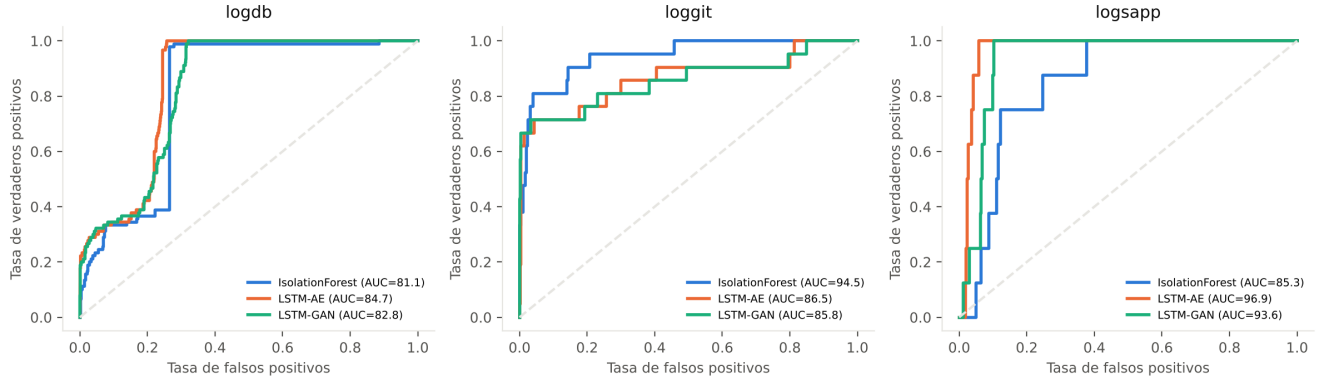
Contraste de hipótesis (prueba de Wilcoxon para muestras relacionadas)

Hip.	Comparación y ámbito	$\bar{\Delta}$	p	r
H1 (F1)	AE frente a IF, consolidado (n = 30)	+8.50	< .001	.84
	`logdb` / `loggit` / `logsapp`	+ / + / +	.002 / .004 / .002	
H2 (FPR)	AE frente a IF, consolidado	−2.98	< .001	.87
	`logdb` / `loggit` / `logsapp`	− / − / −	.002 / .002 / .002	
H3 (F1)	GAN frente a AE, consolidado	−1.31	.129	.28
	`logdb` / `loggit` / `logsapp`	− / − / −	.492 / .625 / .027	
H3 (AUC)	GAN frente a AE, consolidado	−3.36	< .001	.81
	`logdb` / `loggit` / `logsapp`	− / − / −	.010 / .037 / .002	
Compl. (AUC)	AE frente a IF, consolidado	+2.52	.114	.29
	`logdb` / `loggit` / `logsapp`	+ / − / +	.002 / .002 / .002	

Nota. AE = LSTM-AE; GAN = LSTM–GAN; IF = Isolation Forest. $\bar{\Delta}$ = media de la diferencia AE − IF (H1, H2 y fila complementaria) o GAN − AE (H3), en puntos porcentuales. Los signos por subconjunto indican la dirección del cambio; n = 10 pares en cada uno.

Figura 3

Curvas ROC en el tramo de prueba (puntuaciones promediadas sobre 10 semillas)



- Escaneos de *webshells* `.php` y de `/wp-login.php` contra GitLab, servicio que no ejecuta PHP.
- Sondeos de archivos `.env` y de `cgi-bin` contra el ERP: el LSTM-AE detectó 7 de las 8 ventanas maliciosas de prueba en la mayoría de las semillas, frente a 2 del LSTM–GAN.

Hubo también un caso que el modelo no detectó: una enumeración lenta de bases de datos el 8 de mayo, con unos 630 intentos repartidos en cinco horas. Superó el umbral al inicio, pero el contexto de 100 minutos la absorbió como comportamiento habitual.

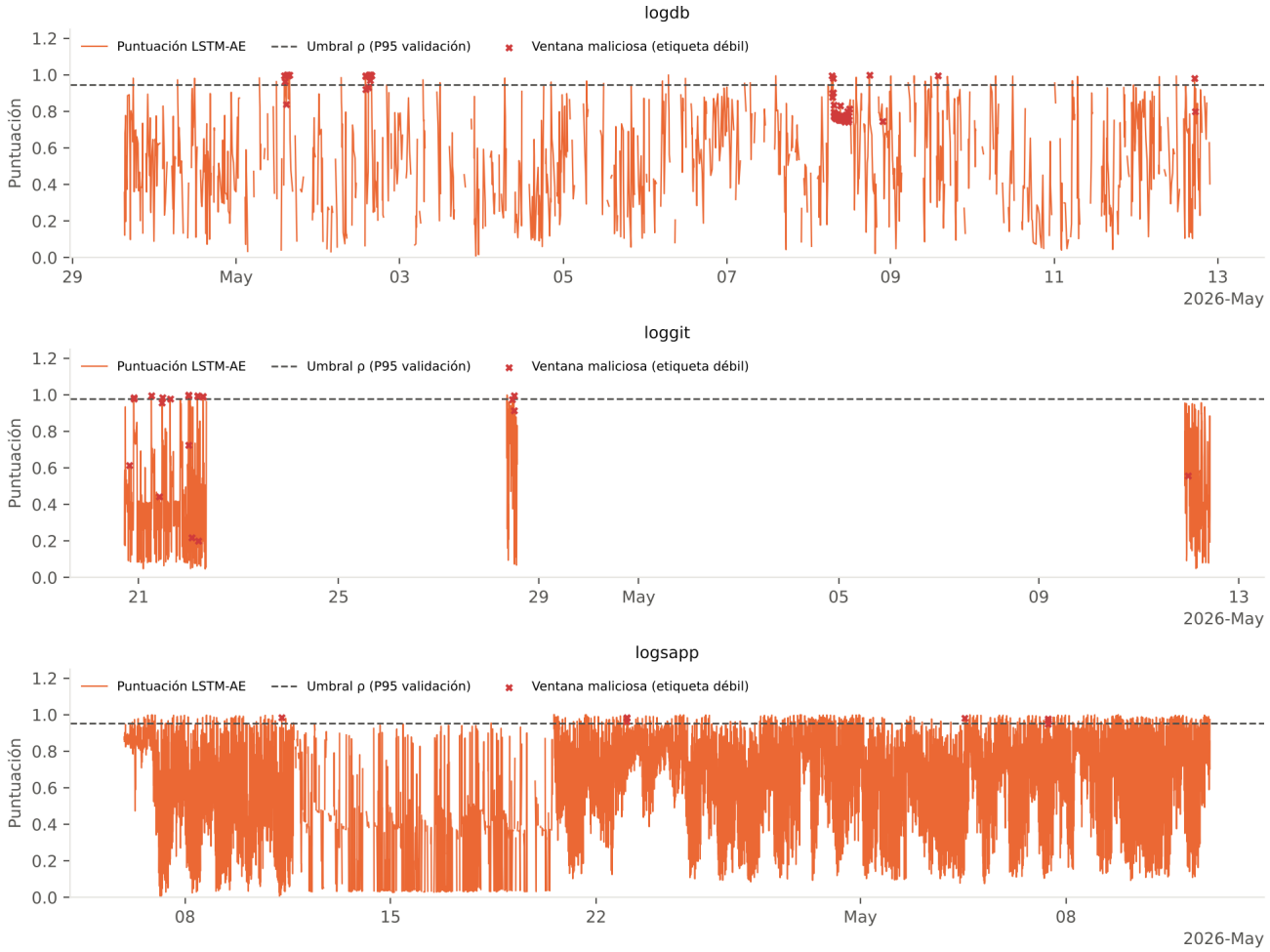
6. Discusión

Los resultados respaldan la hipótesis general. La mejora en la reducción de falsas alarmas y la ganancia de exactitud se explican por la capacidad de la reconstrucción temporal del autocodificador LSTM para modelar el contexto de 100 minutos. Isolation Forest evalúa cada ventana de forma aislada, mientras que el LSTM interpreta el volumen y la composición de los eventos dentro de ese contexto. Esto es especialmente relevante en `logsapp`, cuyo tráfico de *longpolling* varía a lo largo del día. El resultado es coherente con Aslam et al. (2024) y Gorokhov et al. (2023), que también basan la detección en el error de reconstrucción.

La adición de la GAN, en cambio, no aportó valor incremental, a diferencia de lo reportado por Niu et al. (2020) y Qian et al. (2025) en series de sensores. La explicación más probable es que la presencia de anomalías no etiquetadas durante el entrenamiento (64 ventanas maliciosas) contaminó la representación latente que aprende el discriminador, que pasó a tratar también patrones maliciosos como reales; sin etiquetas de entrenamiento, esta explicación no puede verificarse directamente. A ello se suman las dificultades propias del entrenamiento adversarial, como el desvanecimiento del gradiente y el colapso de modos, señaladas por Qian et al. (2025). El riesgo se relaciona, además, con lo mostrado por

Figura 4

Puntuación del LSTM-AE, umbral ρ y ventanas maliciosas en el tramo de prueba



Nota. Las cruces marcan ventanas con etiqueta débil maliciosa. Los huecos corresponden a periodos sin registros.

Liu et al. (2023): una GAN puede generar datos adversariales que envenenan los detectores basados en aprendizaje automático, de modo que los ataques presentes en el entrenamiento no solo degradan el modelo, sino que constituyen una superficie de ataque.

La ventaja de Isolation Forest en el AUC de `loggit`, frente a ambos modelos LSTM, indica que un método clásico puede competir con el aprendizaje profundo cuando las anomalías se manifiestan como cambios de frecuencia, como ocurre con las ráfagas de escaneo web. Esto es coherente con Farzad y Gulliver (2020), cuyos mejores resultados en *logs* se obtuvieron al combinar Isolation Forest con un autocodificador. Del mismo modo, los valores de F1 obtenidos, entre 1.9 y 65.0 %, son muy inferiores a los publicados sobre conjuntos de referencia, por ejemplo el 99.61 % de Nasirzadeh et al. (2025) o el 97.3 % de AUC de Gorokhov et al. (2023) en HDFS. Esta brecha es coherente con Albert (2025), quien mostró que la partición cronológica y la presencia de anomalías en el entrenamiento reducen drásticamente el desempeño reportado, y es comparable al F1 medio de 71.23 % de Zhao et al. (2025) en escenarios de transferencia entre sistemas.

En un contexto más amplio, el hallazgo de mayor valor operativo es la reducción de falsas alarmas sin etiquetas. En `logsapp`, el LSTM-AE emite unas 630 falsas alarmas menos que Isolation Forest durante el periodo de prueba de 30 días, lo que alivia directamente la carga de un equipo de seguridad universitario. El LSTM–GAN reduce todavía más las alertas, unas 845 menos, pero a costa de detectar menos ataques: 2 de las 8 ventanas maliciosas, frente a 7 del LSTM-AE. La omisión de la enumeración lenta ilustra el límite de los modelos de contexto fijo ante anomalías colectivas de baja intensidad, un tipo de anomalía que Nasirzadeh et al. (2025)

tratan de forma específica. A ello se suma la sensibilidad al ruido de los registros reales, documentada por Zhang et al. (2025). Por último, aunque la mejora del F1 es significativa, permanece en el nivel bajo de la escala criterial. Esto indica que, en su estado actual, el modelo es útil como filtro que prioriza alertas más que como detector autónomo.

Limitaciones. Las etiquetas débiles, derivadas de reglas, no equivalen a una verdad de campo y pueden omitir ataques no contemplados en ellas. Se analizaron tres de los cuatro subconjuntos del repositorio, porque `logserver` excedía la capacidad de cómputo disponible. `logsapp` cuenta con muy pocas ventanas maliciosas en el tramo de prueba. Por último, no se realizó una búsqueda de hiperparámetros para evitar ajustar el modelo sobre el tramo de evaluación.

Líneas futuras. Se proponen seis:

1. Calibrar ρ según la tasa esperada de anomalías de cada servicio (Qian et al., 2025; Yokkampon et al., 2022).
2. Emplear contextos multiescala para capturar ataques lentos.
3. Incorporar explicabilidad para el analista e integrar codificadores tipo *Transformer* para anomalías ocultas en varias fuentes (Luo et al., 2025).
4. Transferir el modelo entre subsistemas mediante pseudoetiquetado (Zhao et al., 2025).
5. Fusionar las modalidades semántica y secuencial con transformadores colaborativos (Nasirzadeh et al., 2025).
6. Automatizar sin etiquetas la selección del modelo y de sus hiperparámetros, e incorporar el subconjunto `logserver`.

7. Conclusiones

Se cumplió el propósito de evaluar en qué medida la reconstrucción temporal latente (LSTM-AE) y la adición de un discriminador adversarial (LSTM–GAN) mejoran la detección sin etiquetas de comportamientos maliciosos en las series temporales de *logs* de la infraestructura de servidores universitarios. Las hipótesis específicas se verificaron como sigue:

- **H1 se valida.** Con el LSTM-AE, el F1 aumentó de 25.06 % a 33.56 % ($p < .001$).
- **H2 se valida.** Con el LSTM-AE, la FPR se redujo de 6.27 % a 3.28 % ($p < .001$) y pasó del nivel aceptable al óptimo.
- **H3 se valida.** El discriminador no mejoró el F1 ($p = .129$) y redujo el AUC de 89.04 % a 85.68 % ($p < .001$).

En consecuencia, la hipótesis general se acepta. Se concluye que el autocodificador LSTM-AE es la arquitectura recomendada para el despliegue en entornos universitarios reales, al lograr una reducción del 47.6 % en falsas alarmas frente a la línea base sin incurrir en la inestabilidad ni en los costos computacionales del componente adversarial GAN. La relevancia del estudio reside en haber separado el efecto de ambos componentes sobre 3.2 millones de eventos crudos de tres servicios reales, con partición cronológica, entrenamiento contaminado y contraste estadístico, condiciones más cercanas a la producción que las de los conjuntos de referencia habituales.

Los aportes principales son tres: una representación de ventanas como series semánticas que permite detectar ráfagas de ataque repetitivas, la evidencia de que la reconstrucción temporal, y no el discriminador, explica la mejora, y un repositorio reproducible. En cuanto a la aplicabilidad práctica, un equipo de seguridad universitario puede desplegar el LSTM-AE sin etiquetar datos y obtener cerca de la mitad de falsas alarmas que con la técnica tradicional. Esto responde a la motivación inicial: aprovechar los abundantes registros institucionales sin depender de analistas para etiquetarlos.

Dataset y código fuente

- **Datos crudos:** subconjuntos `logdb`, `loggit` y `logsapp` del repositorio *uni-logs* (https://uni-logs.us-mia-1.linodeobjects.com/list.html).
- **Código y resultados:** los guiones `prep.py`, `experiment.py` y `analyze.py`, el archivo `requirements.txt`, las tablas de resultados y las figuras están disponibles en https://github.com/Arthur100803/ia2-tema7-agentes-adversarios-duales.
- **Ejecución completa:** los comandos están en el archivo `README.md`. Toma unos 105 minutos en una CPU de dos núcleos.

Referencias

Albert, R.-G. (2025). System logs anomaly detection. Are we on the right path? *Applied Artificial Intelligence*, *39*(1), Artículo e2440692. https://doi.org/10.1080/08839514.2024.2440692

Almodovar, C., Sabrina, F., Karimi, S., & Azad, S. (2024). LogFiT: Log anomaly detection using fine-tuned language models. *IEEE Transactions on Network and Service Management*, *21*(2), 1715-1723. https://doi.org/10.1109/TNSM.2024.3358730

Aslam, M. M., Tufail, A., De Silva, L. C., Apong, R. A. A. H. M., & Namoun, A. (2024). An improved autoencoder-based approach for anomaly detection in industrial control systems. *Systems Science & Control Engineering*, *12*(1), Artículo 2334303. https://doi.org/10.1080/21642583.2024.2334303

Duan, X., Ying, S., Yuan, W., Cheng, H., & Yin, X. (2021). A generative adversarial networks for log anomaly detection. *Computer Systems Science and Engineering*, *37*(1), 135-148. https://doi.org/10.32604/csse.2021.014030

Farzad, A., & Gulliver, T. A. (2020). Unsupervised log message anomaly detection. *ICT Express*, *6*(3), 229-237. https://doi.org/10.1016/j.icte.2020.06.003

Gorokhov, O., Petrovskiy, M., Mashechkin, I., & Kazachuk, M. (2023). Fuzzy CNN autoencoder for unsupervised anomaly detection in log data. *Mathematics*, *11*(18), Artículo 3995. https://doi.org/10.3390/math11183995

Liu, Z., Hu, J., Liu, Y., Roy, K., Yuan, X., & Xu, J. (2023). Anomaly-based intrusion on IoT networks using AIGAN—a generative adversarial network. *IEEE Access*, *11*, 91116-91132. https://doi.org/10.1109/ACCESS.2023.3307463

Luo, C., Jiang, H., Zhou, P., Wang, X., Shao, Z., & Sun, W. (2025). A multi-source log hidden anomaly detection method integrating Trans-Encoder and LSTM. *IEEE Access*, *13*, 158123-158136. https://doi.org/10.1109/ACCESS.2025.3556959

Nasirzadeh, M., Tahmoresnezhad, J., & Rashidi-Khazaee, P. (2025). A unified framework for detecting point and collective anomalies in operating system logs via collaborative transformers. *Scientific Reports*, *15*, Artículo 45698. https://doi.org/10.1038/s41598-025-27693-4

Niu, Z., Yu, K., & Wu, X. (2020). LSTM-based VAE-GAN for time-series anomaly detection. *Sensors*, *20*(13), Artículo 3738. https://doi.org/10.3390/s20133738

Qian, C., Tang, W., & Wang, Y. (2025). RGAnomaly: Data reconstruction-based generative adversarial networks for multivariate time series anomaly detection in the Internet of Things. *Future Generation Computer Systems*, *167*, Artículo 107751. https://doi.org/10.1016/j.future.2025.107751

Yokkampon, U., Mowshowitz, A., Chumkamon, S., & Hayashi, E. (2022). Robust unsupervised anomaly detection with variational autoencoder in multivariate time series data. *IEEE Access*, *10*, 57835-57849. https://doi.org/10.1109/ACCESS.2022.3178592

Zhang, H., Zhou, Y., Xu, H., Shi, J., Lin, X., & Gao, Y. (2025). Anomaly detection in virtual machine logs against irrelevant attribute interference. *PLOS ONE*, *20*(1), Artículo e0315897. https://doi.org/10.1371/journal.pone.0315897

Zhao, X., Guo, K., Huang, M., Qiu, S., & Lu, L. (2025). ELFA-Log: Cross-system log anomaly detection via enhanced pseudo-labeling and feature alignment. *Computers*, *14*(7), Artículo 272. https://doi.org/10.3390/computers14070272